up, the parallelism in the algorithm can be increased to match. Such an algorithm is the perfect embodiment of the adage, “Many hands make light work.” For those parts of the algorithm that aren’t embarrassingly parallel, you might be able to divide the algorithm into a fixed (and therefore not scalable) number of parallel tasks. Techniques for dividing tasks between threads are covered in chapter 8.

The second way to use concurrency for performance is to use the available parallelism to solve bigger problems; rather than processing one file at a time, process 2 or 10 or 20, as appropriate. Although this is really just an application of *data parallelism*, by performing the same operation on multiple sets of data concurrently, there’s a different focus. It still takes the same amount of time to process one chunk of data, but now more data can be processed in the same amount of time. Obviously, there are limits to this approach too, and this won’t be beneficial in all cases, but the increase in throughput that comes from such an approach can actually make new things possible—increased resolution in video processing, for example, if different areas of the picture can be processed in parallel.

1.2.3. When not to use concurrency

It’s just as important to know *when not* to use concurrency as it is to know *when* to use it. Fundamentally, the only reason not to use concurrency is when the benefit is not worth the cost. Code using concurrency is harder to understand in many cases, so there’s a direct intellectual cost to writing and maintaining multithreaded code, and the additional complexity can also lead to more bugs. Unless the potential performance gain is large enough or separation of concerns clear enough to justify the additional development time required to get it right and the additional costs associated with maintaining multithreaded code, don’t use concurrency.

Also, the performance gain might not be as large as expected; there’s an inherent overhead associated with launching a thread, because the OS has to allocate the associated kernel resources and stack space and then add the new thread to the scheduler, all of which takes time. If the task being run on the thread is completed quickly, the actual time taken by the task may be dwarfed by the overhead of launching the thread, possibly making the